

Forecasting Inflation in a Big Data Setting Using Neural Networks

Yuhao Huo

yh620

Motivation & Literature

Motivation:

- Forecasting inflation is essential for decision making by firms, investors, and central banks
- Time-series econometrics often relies on linear models, which may suffer from over-fitting & fail to capture non-linearity
- ML methods, such as neural networks, may provide robust predictions in high-dimensional, non-linear settings

Related Literature:

- *Univariate methods:*
 - Lagged inflations as predictors, initially found difficult to beat (Stock & Watson, 2010)
- *ML methods:*
 - With ML and large macro datasets, multivariate models began to achieve systematic gains
 - LASSO: outperforms AR in US CPI inflation forecasting (Medeiros & Mendes 2016)
 - Random forest: outperforms univariate benchmark, LASSO, and Bayesian VAR (Medeiros et al. 2021)
 - Non-linear & non-parametric methods appear to perform better: how about using neural networks?
- *Neural networks:*
 - Both FNN and LSTM improve performance relative to AR (Andresen, 2023 and Paranhos, 2025)
 - LSTM usually achieves better performance than FNN
 - Multivariate FNN and LSTM also outperform their univariate counterparts
 - However, LSTM does not systematically outperform random forest

Data

FRED-MD Database:

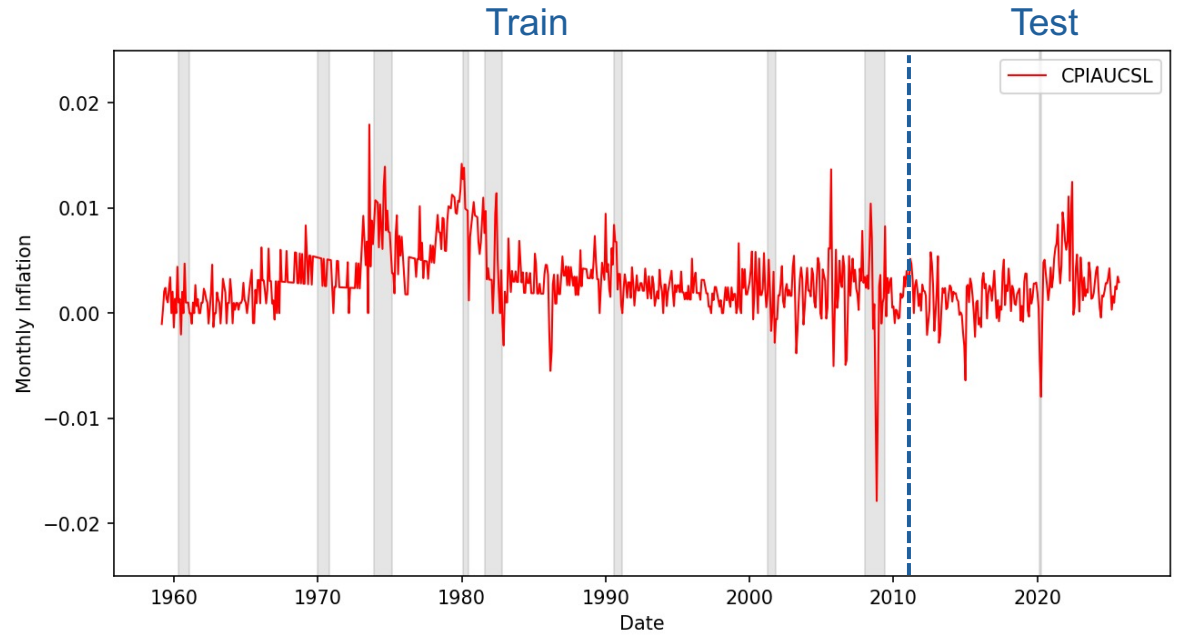
- A monthly macroeconomic database with 126 series
- 802 observations from Mar 1959 to Dec 2025
- **Target variable:** log difference of CPI all-item series
- The other series are transformed to achieve stationary
- Series with > 10% missing values are dropped (4 series)

Train & Test Split:

- **Training sample:** Mar 1959 to Jul 2012 (80% obs)
- **Test sample:** Aug 2012 to Dec 2025

Feature Engineering:

- Missing values (mostly around 0.05%) are imputed using Scikit-learn's **iterative imputer**
- **Principal component analysis** (PCA) is applied to all series to extract 10 principal components because:
 - Macro-economics series are usually correlated
 - The sample has too many dimensions relative to number of observations
- The 10 PCs and the inflation rate are **min-max scaled** to [0, 1], as LSTM performs better at this range
- All transformations are fitted on training set & applied to the full sample



Methodology

Forecasting Methodology:

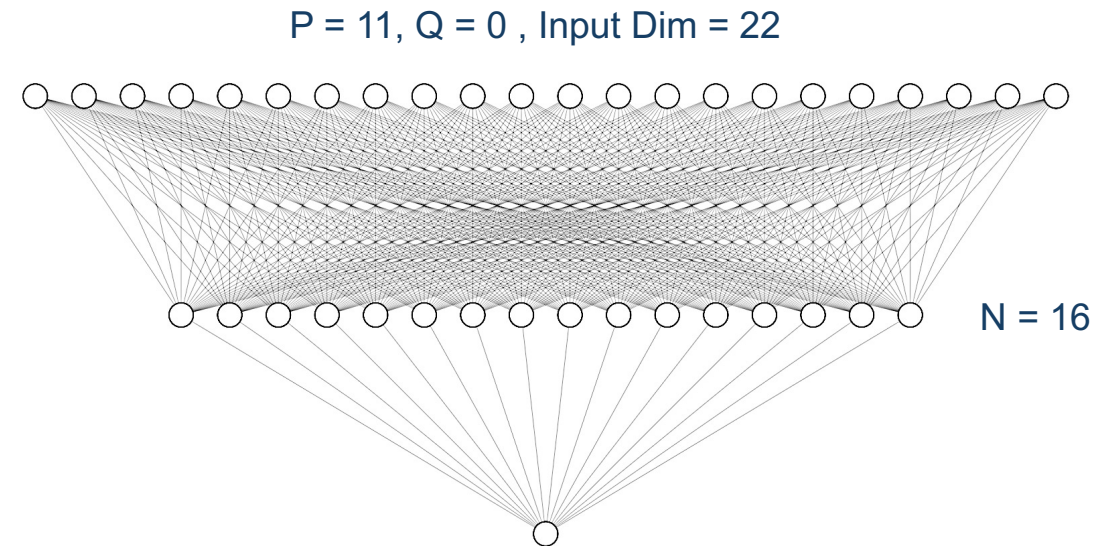
- **Indirect forecasting:**
 - One-step-ahead forecasting,
 - Prediction result is then used as predictor for two-step-ahead forecasting
 - Iterates recursively to reach h-step-ahead forecast
- **Direct forecasting:** estimates a separate model for each forecast horizon h
- Indirect forecasting is more efficient under correct specification, but requires forecasting the covariates
- Therefore, this research adopts direct forecasting and estimates a separate model for each horizon in [1, 3, 6, 12, 24]

Feedforward Neural Network:

- A fully connected neural network with one hidden layer

$$\hat{y}_{t+h} = \phi_0 + \sum_{n=1}^N \phi_n a \left(\sum_{p=0}^P w_{n,p} y_{t-p} + \sum_{k=1}^K \sum_{q=0}^Q \beta_{n,k,q} x_{k,t-q} \right)$$

- y: inflation at period t
- x: k-th principal component at period t
- N: number of nodes
- P: number of lags of inflation included
- Q: number of lags of principal component included



Methodology

Long Short-Term Memory:

- A type of **RNN**: takes a sequence of observations as input, the length of the input sequence is **window size**
- RNN encodes each observation in an input sequence into a **hidden state**
- The hidden state of an observation depends on its features and the previous hidden state
- RNN uses the **last hidden state** of an input sequence to predict the target (h-step-ahead inflation rate)
- RNN suffers from the vanishing gradient problem when the input sequence is long
- LSTM addresses this by introducing **cell state** + **gate mechanisms**

$$f_t = \text{Logit}(\beta_f X_t + w_f H_{t-1} + b_f), \quad i_t = \text{Logit}(\beta_i X_t + w_i H_{t-1} + b_i)$$

$$o_t = \text{Logit}(\beta_o X_t + w_o H_{t-1} + b_o), \quad p_t = \text{Tanh}(\beta_p X_t + w_p H_{t-1} + b_p)$$

$$c_t = (f_t \odot c_{t-1}) + (i_t \odot p_t), \quad H_t = o_t \odot \text{Tanh}(c_t), \quad \hat{y}_{t+h} = w_2 a[w_1 H_t + b_1] + b_2$$

- **f**: forget gate, outputs a proportion in [0, 1], controls how much of the previous cell state pass to current cell state
- **i**: input gate, controls how much of the scaled combination of input is retained in current cell state
- **o**: output gate, controls how much of current cell state is mapped to current hidden state
- **p**: scaled combination of input features and previous hidden state
- **c**: cell state
- **H**: hidden state, passes through a one-layer FNN to form prediction
- **y**: target variable, h-step-ahead inflation rate
- **X**: vector of inflation and PCs at time t

Training & Tuning

Other Benchmark Models:

- AR, ARDL, LASSO, random forest, and LGBM

Training:

- Loss function: mean squared error (MSE)
- For neural networks:
 - Parameters updated by Adam optimization algorithm
 - Batch size = 128
 - Drop out rate = 0.2

Fine-tuning Hyper-parameters

- Hyper-parameters are selected using *grid-search*
- For each set of hyper-parameters, evaluate cross-validation loss, the set with lowest CV loss is optimal
- *CV in time series*:
 - Split training sample into 5 chronological folds
 - For each fold, train model on observations up to that fold and evaluate on subsequent fold
 - Average MSE across folds to obtain CV loss

Table 1: Model Hyper-parameters

Model	Hyper-parameter	Choices	Optimal Value
FNN	Nodes	[8, 12, 16]	12
	Epochs	[100, 300]	100
	Learning Rate	[0.001, 0.0001]	0.001
LSTM	Nodes	[8, 12, 16]	12
	Hidden State Size	[8, 12, 16]	16
	Window Size	[6, 12]	12
AR	Lags	[3, 7, 11, 15, 19, 23]	11
ARDL	Inflation Lags	[3, 7, 11, 15, 19, 23]	7
	Principal Components Lags	[0, 3, 7, 11, 15, 19, 23]	0
LASSO	Alpha	[0.1, 0.02, 0.01, 0.002, 0.001]	0.01
Random Forest	Estimators	[100, 200, 500]	500
	Min Sample Split	[2, 4, 6]	4
GBM	Estimators	[100, 200, 500]	100
	Learning rate	[0.001, 0.005, 0.01, 0.05, 0.1]	0.01
	Max Depth	[-1, 3, 5]	5

Forecasting Performance

Forecasting with Ensembles:

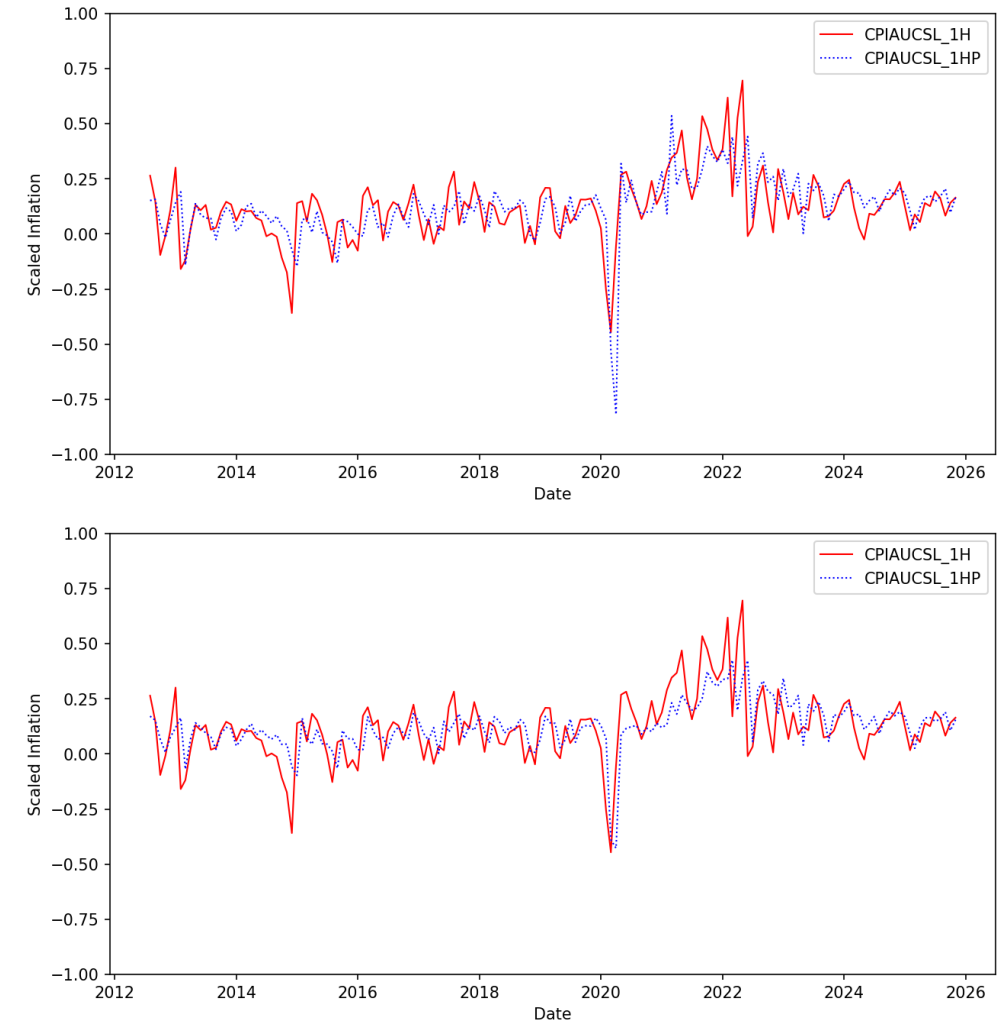
- Neural networks are sensible to initial values (Paranhos, 2025)
- Estimate 20 NNs with different initial values and ensemble them to form predictions for each horizon

Performance:

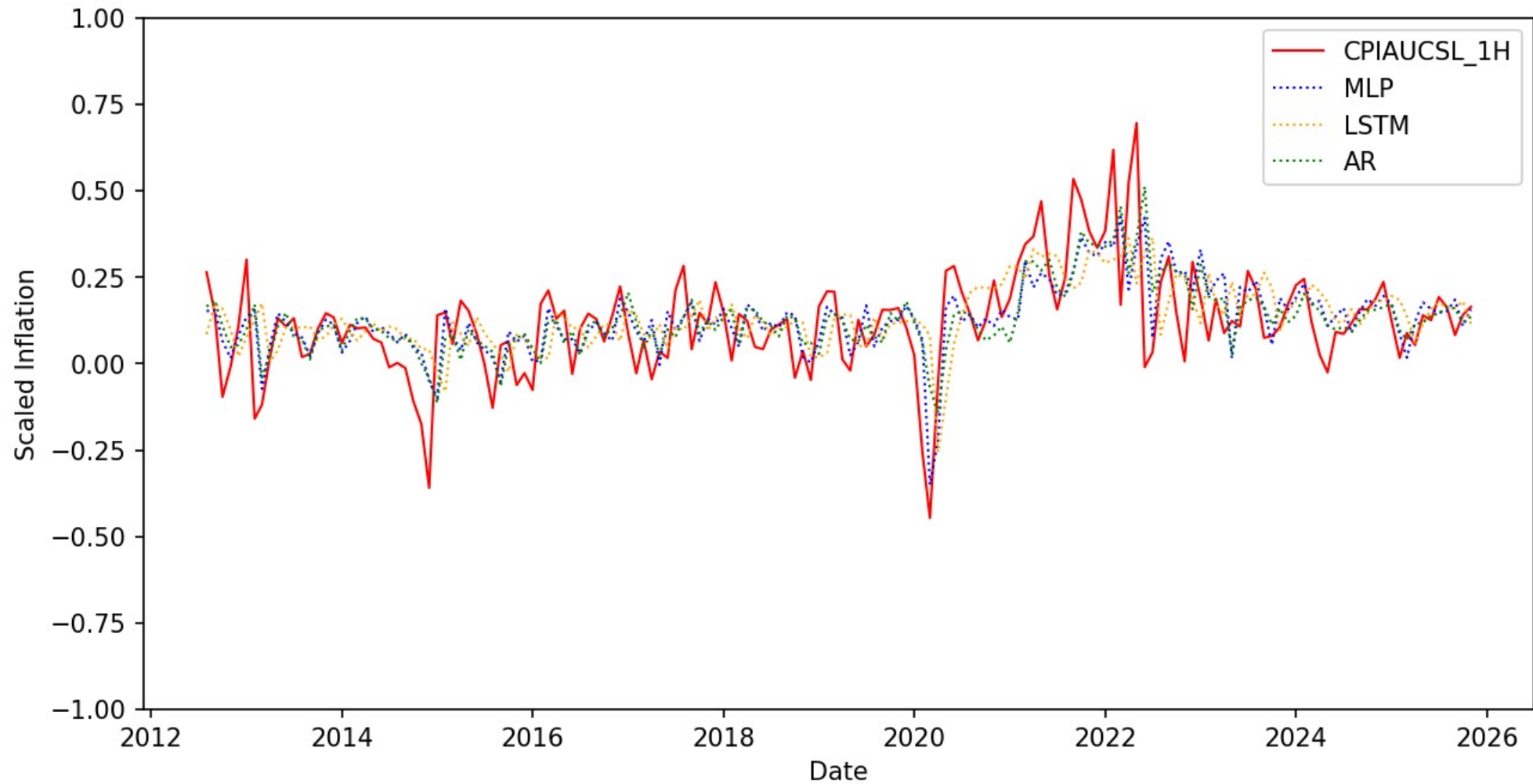
- Evaluated using *RMSE* relative to AR
- Univariate benchmark AR outperforms ARDL and ML methods
- FNN outperforms AR at short forecasting horizons (1 to 6)
- LSTM outperforms AR at longer horizons (3 to 24)
- FNN outperforms LSTM at $h=1$, but performs worse at longer horizons

Table 2: Out-of-sample Forecasting RMSE Relative to AR

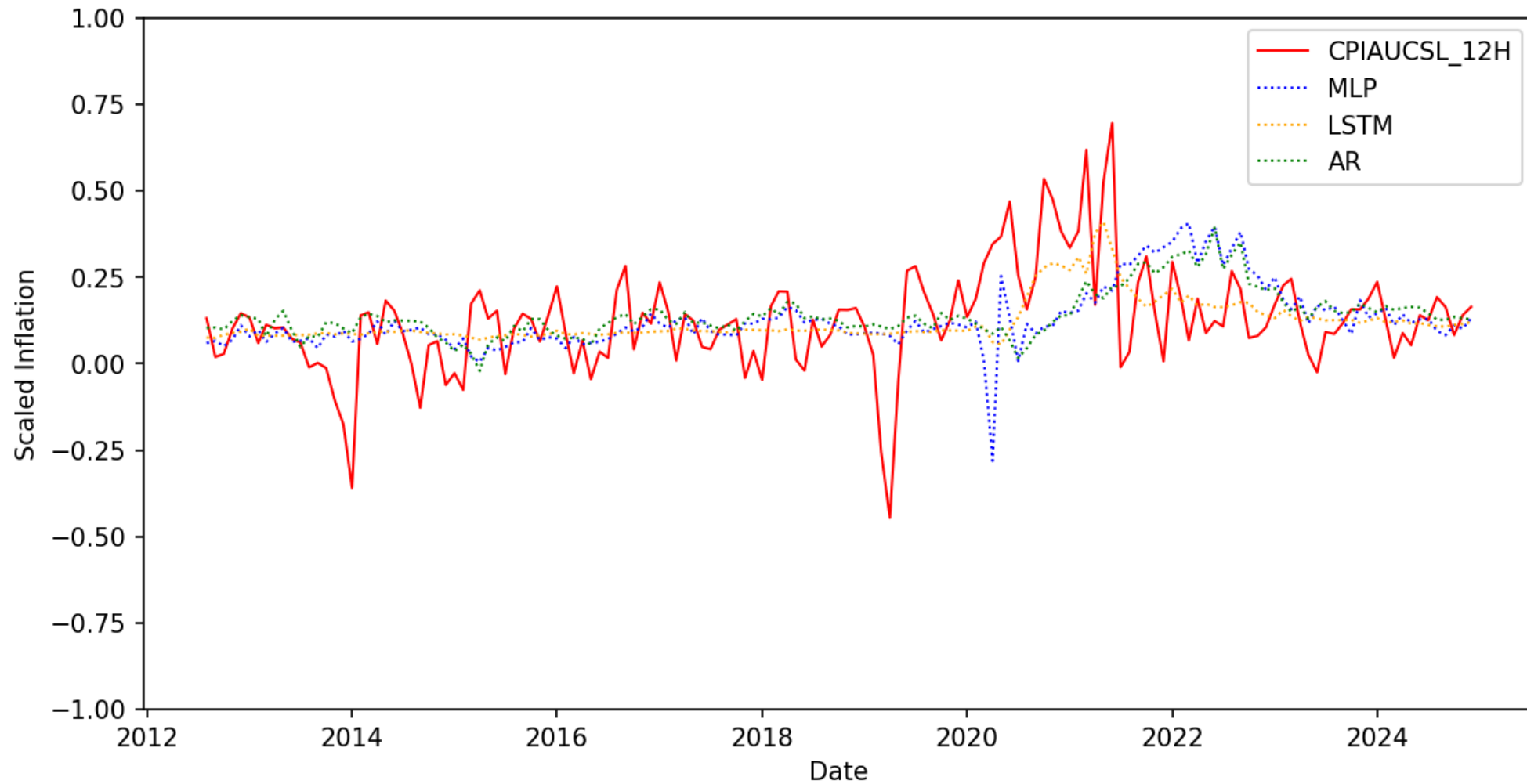
Model	1	3	6	12	24
FNN	93.09	97.95	98.62	106.50	101.03
LSTM	115.31	94.62	89.32	85.10	97.08
ARDL	98.59	109.61	105.66	114.73	107.74
LASSO	142.34	126.44	120.05	116.32	106.43
Random Forest	136.40	119.75	118.41	111.70	103.91
GBM	130.46	116.01	114.32	105.94	97.71



1 Period Ahead



12 Periods Ahead



Discussion

Key Findings:

- A simple FNN outperforms more complex neural network (LSTM) and univariate model in *one-step-ahead* forecasting
- Fine-tuned LSTM perform especially well at *longer horizons*
- FNN and LSTM can be combined to achieve superior forecasting performance across short and long horizons

Limitations:

- Neural networks are not as interpretable as linear models (AR)
- Due to time limitations, this research does not apply grid search to all hyper-parameters
- Cross-validation losses are computed based on one NN initialization for each fold
- Ideally, multiple initializations should be used for each fold to ensure stability

Reference

- [1] Almosova, A., \& Andresen, N. (2023). Nonlinear inflation forecasting with recurrent neural networks. *Journal of Forecasting*, 42(2), 240–259.
- [2] Bredahl Kock, A., \& Teräsvirta, T. (2016). Forecasting Macroeconomic Variables Using Neural Network Models and Three Automated Model Selection Techniques. *Econometric Reviews*, 35(8–10), 1753–1779.
- [3] Marcellino, M. G., Stock, J. H., \& Watson, M. W. (2005). A Comparison of Direct and Iterated Multistep Ar Methods for Forecasting Macroeconomic Time Series (SSRN Scholarly Paper No. 771945). Social Science Research Network.
- [4] Masini, R. P., Medeiros, M. C., \& Mendes, E. F. (2023). Machine learning advances for time series forecasting. *Journal of Economic Surveys*, 37(1), 76–111.
- [5] McCracken, M. W., \& Ng, S. (2016). FRED-MD: A Monthly Database for Macroeconomic Research. *Journal of Business \& Economic Statistics*, 34(4), 574–589.
- [6] Medeiros, M. C., \& Mendes, E. F. (2016). L1-regularization of high-dimensional time-series models with non-Gaussian and heteroskedastic errors. *Journal of Econometrics*, 191(1), 255–271.
- [7] Medeiros, M. C., Vasconcelos, G. F. R., Veiga, Á., \& Zilberman, E. (2021). Forecasting Inflation in a Data-Rich Environment: The Benefits of Machine Learning Methods. *Journal of Business \& Economic Statistics*, 39(1), 98–119.

Reference

[8] Paranhos, L. (2025). Predicting inflation with recurrent neural networks. International Journal of Forecasting. <https://doi.org/10.1016/j.ijforecast.2024.07.010>

[9] Stock, J. H., \& Watson, M. W. (2007). Why Has U.S. Inflation Become Harder to Forecast? Journal of Money, Credit and Banking, 39(s1), 3–33. <https://doi.org/10.1111/j.1538-4616.2007.00014.x>

[10] Stock, J. H., \& Watson, M. W. (2010). Modeling Inflation After the Crisis (Working Paper No. 16488). National Bureau of Economic Research. <https://doi.org/10.3386/w16488>